Workgroup: Independent Submission
Internet-Draft: draft-sibiryakov-ztds-protocol-02
Published: 27 September 2026
Intended Status: Informational
Expires: 31 March 2027
Author: I. Sibiryakov
ZTDS AI Consortium / BrandMeWeb

The Zero-Trust Data Sanitization (ZTDS) Protocol for Frontier Artificial Intelligence Ingestion

Abstract

Zero-Trust Data Sanitization (ZTDS) defines a formal architectural standard and execution protocol for client-side, in-memory de-identification and re-identification across Generative Artificial Intelligence (GenAI), Retrieval-Augmented Generation (RAG), and autonomous agent workflows.

Under ZTDS, sensitive information (including Personally Identifiable Information (PII), Protected Health Information (PHI), financial account numbers, and developer secrets) is intercepted and transformed into synthetic surrogate tokens strictly within volatile memory (RAM) of the originating client or private host node before network serialization.

This protocol specification formalizes the threat model, the four core invariants, surrogate token syntaxes, cryptographic transport handoffs, and verification procedures required for interoperable, zero-subprocessor implementations.

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at https://datatracker.ietf.org/drafts/current/.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 31 March 2027.

Copyright Notice

Copyright (c) 2026 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (https://trustee.ietf.org/license-info) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document.

Table of Contents

1. Introduction
2. Conventions and Terminology
3. Threat Model and Assumptions
4. Mathematical Specification of Protocol
5. The Four Protocol Invariants
   5.1. Invariant 1: Volatile Memory Boundary (RAM-Only Isolation)
   5.2. Invariant 2: Deterministic Reversible Tokenization
   5.3. Invariant 3: Zero Outgoing Network Transmission (Airplane Mode Standard)
   5.4. Invariant 4: Zero Sub-Processor Chain
6. Surrogate Token Taxonomy
7. Cryptographic Transport Handoff
8. Conformance Attestation Architecture
9. IANA Considerations
10. Security Considerations
    10.1. Memory Dumps and Side-Channel Attacks
    10.2. Adversarial Prompt Injection
    10.3. Surrogate Collision and Entropy
11. Privacy and Regulatory Considerations
12. Implementation Status
    12.1. PrivacyScrubber Commercial Reference Implementation
    12.2. Headless Developer SDK (@privacyscrubber/sdk)
    12.3. Model Context Protocol Server (@privacyscrubber/mcp-server)
    12.4. Zero-Trust Conformance Auditor (ztds-audit)
13. References
    13.1. Normative References
    13.2. Informative References

Author's Address

1. Introduction

The proliferation of Generative Artificial Intelligence (GenAI) and Large Language Model (LLM) APIs has created severe data exposure risks. Organizations transmitting unmasked corporate documents, healthcare records, source code, and customer communications over the Public Internet risk compromising sensitive information during model training, telemetry harvesting, or third-party cloud breaches.

Prior approaches have relied on two predominant architectures, each with critical shortcomings:

1. **Network Gateways and Total Blockades:** Prohibiting corporate AI access entirely. This strategy induces widespread employee circumvention ("Shadow AI"), leading to uncontrolled cleartext data exposure.
2. **Cloud-Based Data Loss Prevention (Cloud DLP) Proxies:** Intercepting and inspecting WAN traffic via intermediate multi-tenant cloud services. This introduces significant network round-trip latencies (typically 150ms to 350ms, as empirically measured in [ZTDS-LATENCY]), establishes a single point of data compromise, and introduces an additional Data Processor under European Union General Data Protection Regulation (GDPR) Article 28, mandating cumbersome Data Processing Agreements (DPAs).

The Zero-Trust Data Sanitization (ZTDS) protocol shifts the security perimeter directly into volatile memory (RAM) of the local execution context. No sensitive data may be serialized into network sockets prior to mathematical de-identification. This document specifies the normative requirements for conformant ZTDS implementations.

2. Conventions and Terminology

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in BCP 14 [RFC2119] [RFC8174] when, and only when, they appear in all capitals, as shown here.

Cleartext Entity (S): A sensitive substring matching regulatory or organizational classification rules (e.g., patient name, credit card number, API secret).

Surrogate Token (M): A structured, synthetic replacement token preserving semantic and syntactic properties without revealing cleartext.

Session Mapping Dictionary (R): An ephemeral table linking surrogate tokens to cleartext entities, held exclusively in volatile memory.

Airplane Mode Standard: An empirical test confirming that sanitization and inverse restoration execute with zero network packets emitted across all interfaces.

3. Threat Model and Assumptions

The ZTDS threat model operates under the following security assumptions:

- **Untrusted Transport:** The Wide Area Network (WAN) and all intermediary proxy servers are assumed to be monitored or subject to lawful/unlawful interception.
- **Untrusted Remote Model:** The upstream AI model provider (e.g., public LLM API endpoint) is treated as an untrusted third party that MUST NOT receive cleartext sensitive data, preventing training set ingestion or prompt logging.

- **Semi-Trusted Host Application:** The host execution environment (e.g., browser tab, container, or IDE process) is trusted for local volatile execution, but secondary storage (disk, browser databases, unencrypted telemetry caches) is treated as an insecure persistent boundary.

The primary objective of ZTDS is guaranteeing:

```
Egress(S) == 0 bytes
```

Where S represents the set of all sensitive entities detected within an input prompt or payload.

4. Mathematical Specification of Protocol

Let an input prompt or agent payload P be a sequence of tokens composed of non-sensitive tokens U and sensitive tokens S:

```
P = U union S, where U intersect S = empty_set
```

Where S = {s_1, s_2, ..., s_k} represents discrete sensitive entity instances.

A conformant ZTDS transformation T operates locally as follows:

1. An in-memory transformation T: S -> M is computed locally: M = {m_1, m_2, ..., m_k}, where each m_i is a structured surrogate token.
2. The ephemeral reverse dictionary R = {(m_i, s_i)} is allocated strictly in volatile RAM scoped to the active execution context.
3. The payload serialized and dispatched over the network contains strictly U union M.
4. Upon receipt of the AI inference response P_out containing surrogate tokens M, the local inverse transformation T^(-1): M -> S restores the cleartext entities exclusively on the local presentation layer: P_final = T^(-1)(P_out, R).
5. Upon process termination or session destruction, R MUST be purged from memory.

5. The Four Protocol Invariants

Every compliant implementation of the ZTDS protocol MUST satisfy four normative invariants, with formal mathematical proofs detailed in [ZTDS-PROOF]:

5.1. Invariant 1: Volatile Memory Boundary (RAM-Only Isolation)

Under no circumstances SHALL cleartext entity values S or mapping pairs R be persisted to secondary storage. Implementations MUST NOT write session mapping state to:

- Browser storage subsystems: localStorage, sessionStorage, IndexedDB, document.cookie.
- Host non-volatile storage: temporary files, swap space, disk caches, or crash log dumps.
- Application logging frameworks: debug logs, operational analytics, or tracing pipelines.

In multi-threaded or browser environments, mapping dictionaries MUST be strictly process-isolated or tabId-scoped to prevent cross-session leakage.

5.2. Invariant 2: Deterministic Reversible Tokenization

Surrogate tokens MUST preserve the syntactic category and semantic spacing of the original entity to prevent LLM reasoning degradation. The transformation MUST guarantee:

```
T^(-1)( T(S) ) == S
```

The sanitization engine MUST execute with a deterministic latency bound: Latency(T) < 2.0 ms for standard payloads up to 15,000 characters on standard computing architectures.

5.3. Invariant 3: Zero Outgoing Network Transmission (Airplane Mode Standard)

The sanitization and de-tokenization mechanisms MUST be completely self-contained within local runtime code. Implementations MUST be verifiable via the 5-step Airplane Mode Audit:

1. Ingest sensitive payload into application memory.
2. Sever all physical and virtual network interfaces.
3. Execute sanitization transformation T.
4. Execute reverse re-identification transformation T^(-1).
5. Confirm 100% functional execution with 0 socket errors and 0 bytes egressed.

5.4. Invariant 4: Zero Sub-Processor Chain

The sanitization engine MUST operate as a pure computational utility on the host device. It MUST NOT transmit telemetry, telemetry pings, entity frequencies, or usage metrics to external collection servers. This exclusion relieves the deployment from third-party Data Processing Agreements under European Union GDPR Article 28.

6. Surrogate Token Taxonomy

Surrogate tokens follow standardized bracketed notation to ensure cross-vendor compatibility:

Entity Class	Canonical Token Syntax	Example Transformation
Personal Name	`[NAME_N]`	John Doe -> `[NAME_1]`
Email Address	`[EMAIL_N]`	john@example.com -> `[EMAIL_1]`
Phone Number	`[PHONE_N]`	+1-555-0199 -> `[PHONE_1]`
National ID / SSN	`[ID_N]` / `[SSN_N]`	123-45-6789 -> `[SSN_1]`
Financial / IBAN	`[IBAN_N]` / `[CARD_N]`	GB29XAAA... -> `[IBAN_1]`
Protected Health / PHI	`[MRN_N]` / `[PATIENT_N]`	MRN-90123 -> `[MRN_1]`
Developer Secrets	`[SECRET_KEY_N]`	sk-ant-... -> `[SECRET_KEY_1]`
IP / Host Address	`[IP_ADDR_N]`	192.0.2.1 -> `[IP_ADDR_1]`

Table 1: Standardized Surrogate Token Notation

7. Cryptographic Transport Handoff

When collaborative agent swarms or multi-node enterprise pipelines require transferring mapping state across trust boundaries, mapping dictionaries MUST be encrypted prior to transit:

- **Key Derivation:** Argon2id as specified in [RFC9106] with a minimum of 19 MiB memory cost and 2 iterations.
- **Authenticated Symmetric Encryption:** XChaCha20-Poly1305 with a 24-byte (192-bit) cryptographically secure random nonce generated via a CSPRNG, conforming to [RFC8439] principles.
- **Zero Relay Custody:** Coordination relays act exclusively as blind encrypted conduits with zero mathematical capability to derive keys or inspect mapping pairs.

8. Conformance Attestation Architecture

Conformant implementations MAY generate cryptographically signed Conformance Certificates:

```
ZTDS-CERT-v1.{payload_b64url}.{signature_b64url}
```

Signatures are generated using Ed25519 as specified in [RFC8032]. The payload encompasses a canonicalized JSON structure containing the SHA-256 tree hash of the audited source code, invariant evaluation statuses, and a 365-day validity window.

9. IANA Considerations

This document has no IANA actions.

10. Security Considerations

10.1. Memory Dumps and Side-Channel Attacks

While ZTDS eliminates WAN exposure, local volatile memory remains vulnerable to process inspection if the underlying operating system is compromised. Implementations deployed in high-assurance environments SHOULD utilize hardware-isolated secure enclaves (e.g., AWS Nitro Enclaves or hardware WebAssembly sandboxes) to prevent OS kernel memory dumps from harvesting ephemeral mapping tables.

10.2. Adversarial Prompt Injection

Adversarial prompts designed to trick the LLM into printing mapping dictionaries are mitigated by the architecture: the remote LLM NEVER receives the mapping dictionary R. It only observes surrogate tokens [TYPE_N] as atomic entities.

10.3. Surrogate Collision and Entropy

Surrogate token assignment MUST be deterministic within a given session to ensure co-reference resolution (e.g., the same person mentioned five times receives the identical [NAME_1] surrogate across all mentions), while maintaining fresh indexing per session.

11. Privacy and Regulatory Considerations

By eliminating intermediate data processors, ZTDS satisfies European Union GDPR Article 28 exemption requirements, Article 17 Right to be Forgotten within vector databases (embeddings contain zero extractable cleartext PII), and HIPAA 45 CFR Section 164.514(b) Safe Harbor de-identification rules, as analyzed in [ZTDS-LEGAL].

12. Implementation Status

[Note to RFC Editor: This section MUST be removed before the publication of this document as an RFC, in accordance with [RFC7942].]

This section records the status of known implementations of the protocol defined by this specification at the time of posting of this Internet-Draft, and is based on a proposal described in [RFC7942]. The description of implementations in this section is intended to assist the IETF in its decision processes in creating and maturing specifications. Please note that the listing of any individual implementation here does not imply endorsement by the IETF. Furthermore, no effort has been spent to verify the information presented here that was supplied by IETF contributors. This is not a complete list of all implementations to date. Please send information about any other implementations to the author.

12.1. PrivacyScrubber Commercial Reference Implementation

- **Organization:** BrandMeWeb / PrivacyScrubber (https://privacyscrubber.com)
- **Description:** Pioneer certified commercial implementation of the ZTDS standard. Provides 100% client-side in-memory de-identification across web, browser extensions, autonomous agent IDEs, and backend microservices.
- **Web Application & Sandbox:** Pure client-side single-page application executing inside browser RAM with zero telemetry pings or cloud proxy egress.
- **Browser Extension (Manifest V3):** In-DOM prompt interceptor supporting frontier model interfaces (ChatGPT, Claude, Gemini, DeepSeek, Grok, Copilot, Perplexity, Kimi, Qwen, Zendesk) with zero remote script execution.
- **Cryptographic Handoff Protocol (V5):** Inter-client encrypted session transfer using Argon2id key derivation and XChaCha20-Poly1305 AEAD with framing byte 0x05 and direct zero-knowledge payload links (< 1,850 characters).
- **Licensing:** Commercial and Free Community tiers.
- **Contact:** ilya@brandmeweb.com

12.2. Headless Developer SDK (@privacyscrubber/sdk)

- **Package:** @privacyscrubber/sdk v2.3.3 (npm: https://www.npmjs.com/package/@privacyscrubber/sdk)
- **Description:** Zero-dependency Node.js and WebAssembly (WASM) headless runtime library for embedding deterministic in-memory data sanitization into backend ETL pipelines, LangChain, LlamaIndex, and RAG vector ingestion.
- **Performance:** Sub-millisecond (< 1 ms) in-memory transformation latency; zero outbound socket bindings.
- **Coverage:** 30 specialized industry profiles covering financial, healthcare, legal, engineering, and developer secret taxonomies with zero baseline cannibalization.

- **Status:** Production release v2.3.3 published October 2026.

12.3. Model Context Protocol Server (@privacyscrubber/mcp-server)

- **Package:** @privacyscrubber/mcp-server v2.3.3 (npm: https://www.npmjs.com/package/@privacyscrubber/mcp-server)
- **Description:** Stdio-based Model Context Protocol (MCP) server daemon providing local air-gapped data sanitization for AI IDE agents (Cursor, Windsurf, Claude Desktop, VS Code).
- **Protocol Binding:** Implements JSON-RPC 2.0 stdio transport with multi-agent session isolation, FIFO stream de-tokenization, and zero intermediate network egress.
- **Status:** Production release v2.3.3 published October 2026.

12.4. Zero-Trust Conformance Auditor (ztds-audit)

- **Package:** ztds-audit (npx ztds-audit / https://ztds.ai)
- **Description:** Automated command-line conformance testing utility that audits client applications against the four ZTDS invariants, verifying zero outgoing network packets during transformation under simulated air-gapped network conditions.
- **Licensing:** Open-source / vendor-neutral.

13. References

13.1. Normative References

[RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, March 1997, <https://www.rfc-editor.org/info/rfc2119>.

[RFC8174] Leiba, B., "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words", BCP 14, RFC 8174, May 2017, <https://www.rfc-editor.org/info/rfc8174>.

[RFC8032] Josefsson, S. and I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA)", RFC 8032, January 2017, <https://www.rfc-editor.org/info/rfc8032>.

[RFC8439] Nir, Y. and A. Langley, "ChaCha20 and Poly1305 for IETF Protocols", RFC 8439, June 2018, <https://www.rfc-editor.org/info/rfc8439>.

[RFC9106] Biryukov, A., Dinu, D., Khovratovich, D., and S. Kiviharju, "Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications", RFC 9106, September 2021, <https://www.rfc-editor.org/info/rfc9106>.

13.2. Informative References

[RFC7942] Sheffer, Y. and A. Farrel, "Improving Awareness of Running Code: The Implementation Status Section", BCP 205, RFC 7942, July 2016, <https://www.rfc-editor.org/info/rfc7942>.

[ZTDS-PROOF] Sibiryakov, I., "Zero-Trust Data Sanitization (ZTDS): Mathematical Proof and Operational Framework for Client-Side AI Prompt Privacy", DOI 10.5281/zenodo.22058770, 2026, <https://doi.org/10.5281/zenodo.22058770>.

[ZTDS-LATENCY] Sibiryakov, I., "Empirical Latency Benchmark: In-Memory Client RAM Masking vs. Cloud DLP Proxies", DOI 10.17605/OSF.IO/5BYJF, 2026, <https://doi.org/10.17605/OSF.IO/5BYJF>.

[ZTDS-LEGAL] Sibiryakov, I., "Enterprise Compliance Under the EU AI Act and UK GDPR via Ephemeral Client-Side Sanitization", SSRN 7335581, 2026, <https://papers.ssrn.com/sol3/papers.cfm?abstract_id=7335581>.

Author's Address

Ilya Sibiryakov
ZTDS AI Consortium / BrandMeWeb
Email: ilya@brandmeweb.com
URI: https://ztds.ai